

Thaghrawy — Theoretical Proposal vs. Current Implementation

Cairo University · Faculty of Computers and Artificial Intelligence

Academic Year 2025–2026

Thaghrawy — Theoretical Proposal vs. Current Implementation

Cairo University — Faculty of Computers and Artificial Intelligence Department of Information Technology — Academic Year 2025–2026

Project: Thaghrawy — An AI-Powered Autonomous Penetration Testing Assistant **Supervisor:** Prof. Haitham S. Hamza **Team:** Loay Ahmed Badea (20220259) · Youssef Ali Mohamed (20230644) · Yehia Mohamed Othman (20230646) · Mohamed Abd El-Nasser (20220295) · Omar Ayman Mesbah (20220223) · Belal Mohamed Youness (20220087)

1. Purpose of this Document

This report compares the **theoretical proposal** submitted in Term 1 (*Thaghrawy_Project.pdf* — Chapters I–V) against the **system actually built** to date. It is written for the supervisory committee to (a) see what was delivered against what was promised, (b) understand the rationale behind every engineering divergence, and (c) review an honest account of what remains. Where the proposal and the implementation differ, the difference is deliberate and justified — not an omission.

Headline status: the system is a working, tested prototype (**493 automated tests passing**, ~29 real security tools wrapped, dual PDF reporting, a live benchmark harness, and a full human-in-the-loop collaboration layer). The largest divergences from the proposal are two well-reasoned

technology pivots (recon language, storage) and one area where the delivered system **exceeds** the proposal (human-in-the-loop).

2. Proposal → Implementation at a Glance

Area	Term 1 Proposal	Built Now	Status
Recon engine language	Go (goroutines / worker pools)	Python wrappers orchestrating real tool binaries (many of which are Go: nuclei, subfinder, naabu)	Changed (§4.1)
Primary storage	PostgreSQL (shared state)	ChromaDB (vector memory) + one JSON file per engagement	Changed (§4.2)
Agent architecture	Planner–Executor–Perceptor (3 agents)	Single ReAct tool-calling loop + human-in-the-loop control channel	Changed + Exceeded (§4.3, §5)
LLM host	LM Studio (local only)	Multi-provider abstraction (Anthropic / OpenAI-compatible / Ollama); local LM Studio is the default	More flexible
Vulnerability detection	Custom heuristic servers (SQLi/XSS/IDOR/SSRF/Auth)	Mature tools wrapped as MCP servers: sqlmap, nuclei, nikto, dalfox (XSS), wapiti (broad OWASP),	Changed (higher quality)

Area	Term 1 Proposal	Built Now	Status
Tool count	~5 planned	hydra ~29 security tools (25 recon/scan + 7 exploit, minus 3 workspace utilities) among ~40 registered agent tools	Exceeded
Reporting	Dual technical/managerial reports	Dual PDF reports (technical: CVSS + repro; executive: DREAD + business impact) via reporting/builder.py + report_server.py	Delivered
DREAD risk model	Specified for managerial report	dread_score field on every Finding; executive report tiebreaks by DREAD within a severity band	Delivered
FR-01 mode control	Recon-only / full-analysis toggle	analysis_mode on Engagement; a recon_only agent never registers exploit tools	Delivered
Human-in-the-loop	A security-layer control concept	Full collaboration system: per-tool approve/reject/edit	Exceeded (\$5)

Area	Term 1 Proposal	Built Now	Status
		gate, phased workflow, curation, human-run-a-tool	
Benchmarks	Harness vs DVWA / Juice Shop (ESR/AST/FP)	Implemented + Detection-Rate metric; run live against Juice Shop (numbers in §6)	Delivered (with caveat)
Automated tests	Not mentioned	493 pytest tests, CI-enforceable	Exceeded
Beyond the proposal	—	Phased enumeration → collaboration → reporting workflow; deterministic auto-ingest of scanner output into findings; training-data export (SFT + DPO preference pairs)	New (§5)

3. Requirements Traceability (proposal → evidence)

Proposal item	Target	Status	Evidence
FR-01 Recon-only vs full-analysis	High	Met	build_filtered_registry(); tests in tests/test_core_tools.py (recon_only excludes exploit tools)

Proposal item	Target	Status	Evidence
FR-02 Dual technical + managerial report	High	Met	reporting/ builder.py, generate_report; live reports in reports/
FR-03 AI integration for analysis	High	Met	core/agent.py ReAct loop + core/llm.py multi-provider
FR-04 Searchable scan/report history	Medium	Met	engagements/ manager.py + ChromaDB semantic search (memory/store.py)
NFR maintainability / documented code	—	Met	Type hints, CONTRIBUTING. md, 493 tests
NFR portability (Docker)	—	Met	docker- compose.yml; image builds with all tools baked in
NFR portability (CSV/JSON export)	—	Met (extended)	Training-data export to JSONL; findings via API
Metric: ESR $\geq 70\%$	Success	Not met with local model	Live: 10% (1/10) — model-bound; see §6
Metric: FP-rate $\leq 15\%$	Success	Not met with local model	Live: 33% (1/3) — one nikto false positive; §6

Proposal item	Target	Status	Evidence
Metric: Detection $\geq$ 8/10 OWASP	Success	Not met with local model	Live: 1/10 (A05); §6
Constraint: no AI model training	Scope	Respected	We <i>export</i> training data; we do not train a model in this project
Constraint: integrate existing tools (no custom tools)	Scope	Respected	All detection uses public tools (sqlmap, nuclei, dalfox, ...)

4. Architecture Pivots and Rationale

4.1 Go recon engine → Python orchestration

The proposal chose Go for the reconnaissance engine, citing goroutine concurrency and the ArsOn Framework as prior art. In practice the dominant cost in tool orchestration is the **wall-clock runtime of the external tools themselves** (an nmap scan, a sqlmap crawl, a nuclei sweep), not orchestration overhead — and several of those tools (nuclei, subfinder, naabu) are already compiled Go binaries. No orchestration-language concurrency primitive makes nuclei finish faster. Python was chosen because the entire LLM-tooling ecosystem (Anthropic/OpenAI SDKs, the MCP SDK, ChromaDB, FastAPI, sentence-transformers) is Python-first, and a single-language codebase is realistically maintainable by a six-student team in the available time. The proposal’s *performance intent* (fast, parallel recon) is still met — by the underlying Go tools — while integration complexity drops sharply.

4.2 PostgreSQL → ChromaDB + JSON

PostgreSQL was proposed as a general shared-state store. The dominant access pattern that emerged is “*have we seen something like this before, across all past engagements?*” — a **semantic-similarity** query, not a relational one. ChromaDB answers this natively with local all-MiniLM-L6-v2 embeddings (fully offline, no pgvector, no migrations). Structured engagement

records, which need only simple CRUD, live in one JSON file per engagement. This removes a whole database dependency from every developer’s machine while directly serving the project’s defining feature — persistent cross-engagement memory.

4.3 Planner–Executor–Perceptor → ReAct + human-in-the-loop

The proposal’s three-agent PEP paradigm was simplified to a single **ReAct** (Reason–Act–Observe) tool-calling loop, which proved sufficient to drive ~29 tools. Crucially, the proposal’s mention of human-in-the-loop as a *security-layer control* was **expanded into a first-class collaboration model** (see §5): rather than the LLM acting alone, the human and the model complement each other. This is the single biggest area where the delivered system goes beyond what was proposed.

5. What Was Added Beyond the Proposal

1. **Human-in-the-loop collaboration.** A per-engagement control channel (`core/control.py`) lets the operator **approve** / **reject** / **edit** each tool call, run tools manually, and curate findings, over a concurrent WebSocket and slash-command protocol. The proposal only named “human-in-the-loop” as a security control; it is now the interaction model.
2. **Phased workflow.** Engagements move enumeration → collaboration → reporting. In the autonomous enumeration phase the agent runs recon and **auto-ingests** the easy findings deterministically (`core/finding_drafts.py`), so results no longer depend on a weak model choosing to save them. It then hands off to the human for the harder, gated work.
3. **Training-data export.** Findings, techniques, and every human approve/reject/edit decision are exportable as fine-tuning datasets — supervised examples and **DPO-style preference pairs** (`training/exporter.py`). Human supervision becomes labelled data that can improve a future model. (This is data *export*, within the proposal’s “no model training” scope.)

- 4. **OWASP coverage + live smoke harness.** Added dalfox (XSS) and wapiti (broad OWASP sweep); a live tool-smoke script drives every registered tool against an owned target to catch real-CLI bugs that mocked unit tests cannot.
-

6. Evaluation: Honest Benchmark Results

The benchmark harness (benchmarks/) computes the proposal’s four metrics — ESR (Exploit Success Rate), AST (Average Steps per Task), FP-rate, and Detection Rate (distinct OWASP Top-10 classes) — against an engagement’s saved findings.

A **live** agent run against OWASP Juice Shop produced:

Metric	Target	Measured (local 9B model)	Result
ESR	≥ 70%	10% (1/10 categories)	Below target
AST	minimise	20 steps / task	—
FP-rate	≤ 15%	33% (1/3 findings)	Below target
Detection	≥ 8/10	1/10 (A05 Security Misconfiguration)	Below target

Separately, the autonomous **enumeration** phase auto-ingested **21 findings** from a single nuclei sweep against Juice Shop, and the phase correctly handed off to collaboration.

Interpretation (important). These numbers are **model-bound, not pipeline-bound**. The development environment runs a weak local 9-billion-parameter model (qwen3.5-9b-heretic via LM Studio) chosen for zero API cost during development. It reconnoitres well but is weak at multi-step tool-use and exploitation, so it reaches mostly misconfiguration-class findings and produced one false positive. The **end-to-end pipeline is validated** — the agent runs tools, saves findings, generates both reports, and scores them — but attaining the proposal’s target metrics depends on model capability. Run with a stronger model (e.g. a frontier hosted model, which the multi-provider abstraction already supports), the same pipeline is expected to approach the

proposal’s targets. We report the measured numbers exactly and treat the shortfall as a documented, model-dependent limitation rather than a pipeline defect.

7. Honest Gap Analysis

Proposed / expected	Status	Note
Dedicated IDOR / SSRF / Auth MCP servers	Partial	ssrf_test/upload_test exist as recon probes; no standalone IDOR server. Broad coverage via wapiti + nuclei instead.
Target metrics reached (ESR/Detection/FP)	Not yet	Model-bound; needs a stronger model (§6).
Authenticated scanning (e.g. DVWA behind login)	Not yet	DVWA 302-redirects to login; scanners run unauthenticated. Human-run-a-tool partially mitigates.
LLM-assist (/suggest, /draft) handlers	Stubbed	Protocol + UI hooks exist; one-shot handlers pending.
Go-based recon engine	Replaced	Deliberate pivot (§4.1).
PostgreSQL	Replaced	Deliberate pivot (§4.2).

8. Talking Points for the Committee

Strengths to highlight

1. ~29 **real** security tools driven by genuine autonomous decisions — not simulations.
2. **Persistent cross-engagement memory** (ChromaDB semantic recall) — uncommon in student work.
3. **Human-in-the-loop collaboration** — approve/reject/edit, phased hand-off, curation.

4. **Dual-audience reporting** with CVSS + DREAD — matches real-world reporting practice.
5. A **training-data flywheel** — human supervision becomes fine-tuning data (SFT + DPO pairs).
6. **493-test** CI-grade suite and a live tool-smoke harness that catches real-CLI bugs.

Anticipated questions

- *Why Python instead of Go?* The bottleneck is the tools' own runtime, and several are already Go binaries; Python gives a first-class LLM ecosystem and a maintainable single codebase (§4.1).
- *Why ChromaDB instead of PostgreSQL?* The core query is semantic similarity; ChromaDB does it natively and offline, with no extra database to operate (§4.2).
- *Did you hit your target metrics?* The pipeline is validated end-to-end; the measured numbers are below target **because of the weak local development model**, and are expected to improve substantially with a stronger model (§6). We report them honestly.
- *How is this different from PentestGPT?* PentestGPT advises a human who still runs every tool; Thaghrawy invokes tools autonomously and, in collaboration mode, asks the human to approve/steer — a two-way partnership, plus persistent memory and a training loop.
- *You said “no model training” — what’s the training export?* We export data ready for fine-tuning; we do not train a model within this project. The export respects the scope constraint while enabling future improvement.

Companion documents: the full Final Documentation (Appendix B.2 structure) and the updated Appendix A proposal form accompany this report.